FIRE: THE FIRST-YEAR INNOVATION & RESEARCH EXPERIENCE

QUANTUM MACHINE LEARNING

Teaching Physics to Machine Learning Models

Enhancing Quantum Classical PINNs With Trainable Angle Embeddings

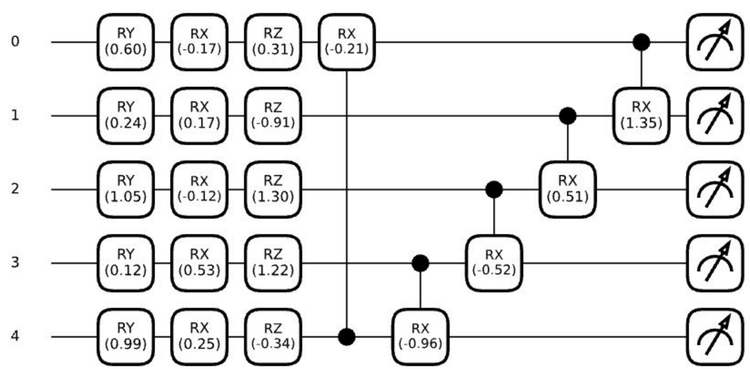
Aryahvishwa Babu, Pranav Bykampadi, Arav Juneja

Introduction

Solving partial differential equations (PDEs)- especially high-dimensional, nonlinear problems- is computationally expensive with classical methods. Physics-Informed Neural Networks (PINNs) help by baking physical laws into the loss, and Quantum PINNs (QPINNs) further add quantum layers to capture more complex PDE behavior, but most existing QPINN implementations rely on continuous-variable photonic systems that aren't available on commercial hardware.

Our work extends this line to discrete-variable quantum hardware by adapting the QCPINN framework to run on IonQ's qubit-based processors, enabling practical testing of quantum-classical hybrid PINNs on real devices. Within this DV setting, we investigate whether trainable embeddings- a small MLP (2→32→32→4) applied before angle encoding- can improve the accuracy and convergence of QCPINNs, comparing a standard DV-QCPINN (direct angle encoding) against our TE-QCPINN variant.

Circuit



The circuit uses 5 qubits with a sim_circ_19 ansatz, which applies RX(θ_i) and RZ(φ_i) single-qubit rotations followed by CRX(α_i) controlled-rotation gates for entanglement, where θ, φ, and α are trainable parameters learned during optimization. The baseline DV model uses AngleEmbedding to directly encode the spatial coordinates (x, y) into fixed RX rotation angles. The trainable embedding (TE) variant replaces this with learnable RY($\beta_i \cdot x$) gates, where the basis coefficients β_i are generated by a 2→32→32→5 MLP that maps the input (x, y) to five learned scaling factors (β_1, β_2, β_3, β_4, β_5), allowing the circuit to optimize how input features are encoded into the quantum state. Both circuits use PennyLane's default.qubit for training with analytic gradients (shots=None) and can be validated on IonQ's ionq_simulator and forte-1 backends.

Equations

Helmholtz equation

$$\Delta u(x,y) + k^2 u(x,y) = f(x,y) \qquad (x,y) \in \Omega$$
$$u(x,y) = h(x,y) \qquad (x,y) \in \Gamma_0$$

Loss function used for Helmholtz

$$\mathcal{L}(\theta) = \min_\theta\big(\lambda_1 \|\mathcal{L}_{\text{phy}}(\theta)\|_\Omega + \lambda_2 \|\mathcal{L}_{\text{bc}}(\theta)\|_{\Gamma_0}\big)$$
$$= \min_\theta\big(\lambda_1 \text{MSE}(\| \underbrace{u_{\theta_{xx}}(x,y) + u_{\theta_{yy}}(x,y) + \alpha u_\theta(x,y)}_{\text{PDE residual}} \|_\Omega) + \text{MSE}(\lambda_2 \| \underbrace{u(x,y) - u_\theta(x,y)}_{\text{Boundary condition}} \|_{\Gamma_0})\big)$$

The Helmholtz equation models how waves behave in space at a fixed frequency, making it a core tool for understanding and simulating resonance, scattering, and energy propagation in physics.

We define two loss terms:

Physics Residual (loss_r) checks if the model satisfies the equation inside the domain and measures how well the network follows the true differential equation.

Boundary Loss (loss_bc) checks if the model matches the required boundary values The physics loss and enforces the fixed values the solution must take on the edges.

This is a key differentiating factor which is only applicable for our Physics Informed Neural Network

Training minimizes a weighted sum of both losses, ensuring the model learns a solution that respects the physics and the boundary conditions.

We use the identical loss formulation from the original QCPINN paper [2].

Method/approach

1) Literature Review & Reproduction: We studied the QCPINN framework (Martínez-Cifuentes et al., CV-based) and TE-QPINN architectures (trainable embedding layers). We reproduced baseline TE-QPINN results on PennyLane (2–8 qubits) to verify implementation before hardware deployment.

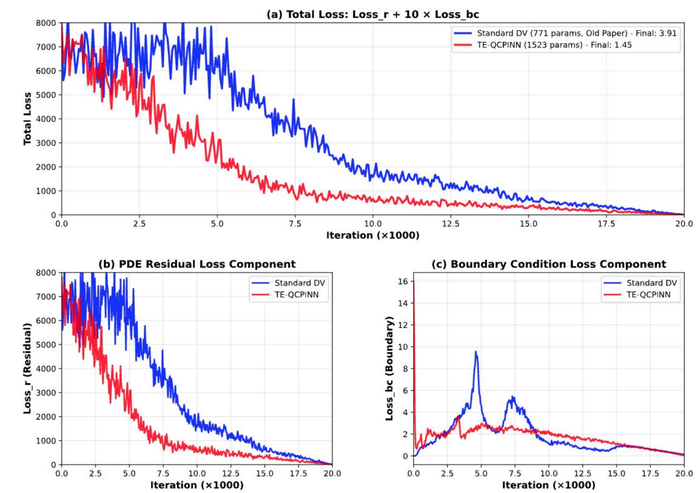
2) Hardware Adaptation: We modified the QCPINN codebase to replace Strawberry Fields CV circuits with discrete-variable qubit circuits for IonQ hardware. Using PennyLane + Qiskit-IonQ, we implemented angle encoding (2→4 features via TE).

3) Training & Validation: We trained the Helmholtz QCPINN on default.qubit for 20,000 epochs (lr=0.001, batch=128, analytic gradients) for both standard DV (461 params) and TE-enhanced (1,224 params) variants.

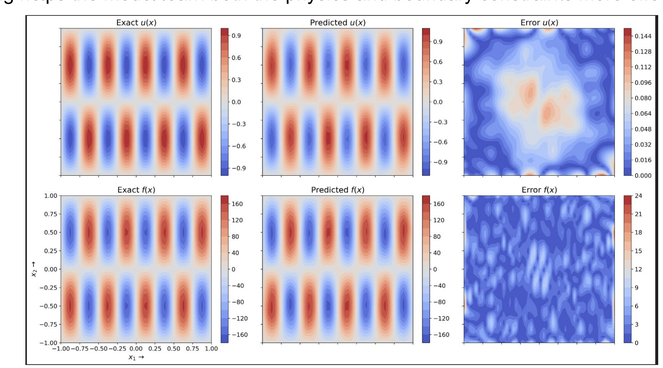
Results

Metric	QCPINN (Old Paper)	TE-QCPINN	Difference
Parameters	771	1,523	+752 (+97.5%)
Initial Loss_r	7.80×10^3	7.96×10^3	Same starting point
Final Loss_r	2.56	0.95	**62.9% better**
Initial Loss_bc	3.40×10^{-2}	1.61×10^1	-
Final Loss_bc	0.135	0.050	**63.0% better**
Initial Total Loss	7.80×10^3	7.96×10^3	-
Final Total Loss	3.91	1.45	**62.9% better**

Results and Future Plans



The local simulation results show that TE-QCPINN converges to a final loss of 1.45 compared to 3.91 for standard QCPINN, demonstrating that trainable embeddings improve solution accuracy. Both Loss_r (PDE residual) and Loss_bc (boundary conditions) decrease faster with TE, showing the trainable encoding helps the model learn both the physics and boundary constraints more effectively as shown above.



Shows Helmholtz equation results after 20,000 training iterations.

Top row: true solution u(x) (the field we want), QCPINN-predicted u(x) and their error—small differences show the model matches the analytic solution well.

Bottom row: true forcing term f(x) (the source term computed from the PDE using the exact u(x)), predicted f(x), and the physics-residual error.

Errors are under 10% for both u(x) and f(x), indicating QCPINNs can solve PDEs accurately even with quantum hardware noise.

We are now ready to test our models on real quantum hardware such as IonQ and IBM, and we are excited to explore how these results advance this emerging area of quantum machine learning.

1. QCPINN PAPER: https://arxiv.org/abs/2503.16678
2. TEPINN PAPER: https://www.nature.com/articles/s41598-025-02959-z
3. Our Code : https://github.com/PranavBykampadi/QCPINN-FIRE